

第087课 实验：让激活和梯度自己说话

实验任务与交付物

你的任务不是给每张图配一句“梯度消失”，而是形成一条完整证据链：写下预测，确认实现，固定对照，读取实测，再写清楚结论边界。实验不需要联网，训练与反传均由NumPy实现。

本课分为三个互补部分：等宽随机网络初始化诊断；真实Wine小样本记忆；简化残差块的传播对照。第一部分没有学习过程，第二部分真正更新参数，第三部分展示残差结构的帮助与限制。最后提交一份记录，包含运行环境、数据哈希、至少六个关键数值、四幅图的解释和练习答案。

请先完成“预测”再看answers.md。第一次运行建议保留默认协议，不要同时修改深度、宽度、种子和学习率。已有experiment-result.json是课程制作时的冻结结果；默认命令把你的结果写入outputs目录，不覆盖原始证据。

0 准备环境与最短运行路线

需要Python 3.11以上，以及requirements.txt中的NumPy与Matplotlib。安装依赖可能需要联网；安装完成后的实验、测试和绘图不调用网络，也不会在线下载数据。

```
python -m pip install -r requirements.txt
export OPENBLAS_NUM_THREADS=1
export OMP_NUM_THREADS=1
export MPLCONFIGDIR="$PWD/outputs/mpl-cache"
export XDG_CACHE_HOME="$PWD/outputs/cache"
python experiment.py --out outputs/result.json
python test_experiment.py --out outputs/test-result.json
python -O test_experiment.py --out outputs/test-result-optimized.json
python plots.py --result outputs/result.json --directory outputs/figures
```

Windows用户可用相应终端语法设置环境变量，或先不设置线程变量运行。线程变量是低资源建议，不是算法定义。不要因为机器上没有中文字体就改变实验；图中文字可换成已安装字体，数值不受影响。冻结图片已经包含中文字体的渲染效果。

依次打开输出JSON，检查propagation、residual与training三个列表。测试结果必须同时满足successful为true、failures为0、errors为0；不能只看文件存在。python -O用于证明测试不依赖可被优化模式删除的裸assert。它不是浮点数高精度模式，也不是速度公平比较的一部分。

1 先认识数据，建立任务边界

data/wine_tiny.json保存30条真实历史记录，每类10条、每条13个原始化学特征。row_ids是UCI Wine随包顺序的零起始行号；labels为0、1、2。原库类别编号与课程的零起始编号在data/README.md中说明。原始数值没有在文件中提前标准化。

样本选择规则在训练之前固定：对每一类取原始行号最前面的10条。训练函数仅用这30条的列均值和列标准差进行标准化。记录有真实来源，并不意味着这个小集合具有代表性；按原始顺序取样尤其不应被当成随机总体抽样。

实验没有独立测试集，这是一项刻意过拟合验收。报告中必须使用“训练准确率”“记忆成功”等称呼，不得写“模型测试准确率100%”。验收条件同时要求准确率100%与平均交叉熵小于0.02，三个种子均报告。

2 手算热身：把统计量分开

给定预激活 $[-3, -1, 1, 3]$ ，应用ReLU得到四个激活。依次计算均值、二阶矩、方差和RMS，随后用moments验证。

```
import numpy as np
import experiment as ex
z = np.array([-3., -1., 1., 3.])
a = ex.activate(z, 'relu')
print(a)
print(ex.moments(a))
```

预测：激活二阶矩是否恰好为输入二阶矩的一半？激活方差是否也减半？为什么这两问答案不同？这里的四点集合严格对称，因此二阶矩对半是精确关系，不是随机误差结论。

进一步取100000个标准正态数作蒙特卡洛检查。固定你自己的新种子，比较ReLU均值与 $1/\sqrt{2\pi}$ 、二阶矩与 $1/2$ 、方差与 $1/2 - 1/(2\pi)$ 。合理抽样误差是正常的；不要把随机估计结果硬改成理论值。

3 检查初始化尺度与矩阵约定

调用weight_std分别计算64→64和64→32层的Xavier与He标准差。写下每个值的平方，区分标准差与方差。再阅读initialize_classifier：权重形状为输入宽度×输出宽度，最后一层始终Xavier。

```
for shape in [(64, 64), (64, 32)]:
    for mode in ['small', 'xavier', 'he', 'large']:
        std = ex.weight_std(*shape, mode)
        print(shape, mode, 'std=', std, 'variance=', std**2)
```

不要把批大小128放进fan_in，也不要因为矩阵在别的框架里按输出×输入保存就照搬这里的轴号。给同一个64→32层画出一名输出神经元接收多少条边，一名输入神经元发出多少条边，比背矩阵形状更可靠。

4 先审计梯度，再相信任何训练结论

打开test_experiment.py，找到三个不同的数值核验：激活导数有限差分、输入向量-雅可比乘积有限差分、分类器所有权重和偏置的有限差分。最后还有独立的残差输入梯度有限差分。它们覆盖的是不同计算环节。

中心差分对参数 θ_i 的近似为

$$\frac{\partial \mathcal{L}}{\partial \theta_i} \approx \frac{\mathcal{L}(\theta_i + \epsilon) - \mathcal{L}(\theta_i - \epsilon)}{2\epsilon}.$$

本课使用float64与 $\epsilon = 10^{-6}$ 。ReLU在零点不可微，所以局部测试避开明显折点；若你修改参数后误差突然增大，先检查正负扰动是否跨过某个ReLU门。数值梯度不是绝对真值，它受舍入误差与截断误差共同影响。

练习时可以临时把反向传播中的权重转置删掉，或把除以批大小改成除两次，再运行测试观察能否抓住错误。完成后必须恢复代码并重跑。不要把故意破坏后的结果当成课程正式结果。

5 初始化探针：预测深度效应

固定宽度64。先手算ReLU配small、Xavier、He、large的单层二阶矩倍率，再将倍率连乘2、8、24次。不要期待每个种子恰好等于预测值；先关注数量级。

```
for mode in ['small', 'xavier', 'he', 'large']:
    for depth in [2, 8, 24]:
        row = ex.probe(depth=depth, width=64, seed=8701, mode=mode)
        print(mode, depth, row['final_second_moment'], row['gradient_rms_ratio'])
```

这里的梯度来自独立随机上游探针，并非分类损失。相同种子对应相同标准正态底稿；改变初始化只改变其缩放。输入和上游探针的随机流与权重流独立。记录控制变量是这个实验最重要的步骤之一。

读取全部三个种子后，用中位数概括并报告最小与最大。不要把最小-最大带标成95%置信区间。为了检验宽度效应，可另做宽度32、128的实验，但那是扩展实验，应与默认64宽度结果分开存档。

6 分布图：读取对数、零点和方向

用plots.py生成四幅图。图2展示尺度随深度变化，图3展示非零值的对数绝对值分布。给图3写清楚三个句子：横轴负数表示什么；零激活去了哪里；为什么同种子、同深度下small、Xavier、He的零比例完全一致。

从probe_samples取得数组，验证图中直方图确实对应末层激活与输入梯度：

```
a, g = ex.probe_samples(depth=24, seed=8701, mode='he')
print('first activation shape', a[0].shape)
print('last activation shape', a[-1].shape)
print('input gradient RMS', ex.moments(g[0])['rms'])
print('last gradient RMS', ex.moments(g[-1])['rms'])
```

a[0]是输入，a[-1]是最后隐藏层；g[0]对输入求导，g[-1]是外部上游探针。列表索引和图层号不是完全一样的概念。若画参数梯度，必须重新命名，不能继续沿用“输入梯度”。

7 饱和与整体梯度可能给出不同线索

比较24层sigmoid/Xavier与tanh/large。对每个组合，记录末层二阶矩、末层小导数比例和输入梯度RMS比。提出预测：“很多单元导数很小”是否足够推出“总梯度一定很小”？“没有极端饱和”是否足够推出“梯度一定能传回来”？

在这个实验中，两句话都不成立。你的解释必须包含权重倍率与多路径传播，而不能只复述数据。进一步检查第一层、中间层和最后一层，思考末层指标为什么不足以概括整条网络。

注意0.01是本课操作性阈值，不是饱和的自然常数。改变阈值会改变比例，因此报告时要一起记录。对ReLU的小导数比例，统一叫关闭门比例，不直接称为tanh式饱和。

8 真正更新参数，验收小样本记忆

运行默认train_tiny或读取完整training结果。三种初始化共享结构、训练数据、学习率、步数和随机底稿；没有为了He单独增加预算。

```
for mode in ['small', 'xavier', 'he']:
    trained = ex.train_tiny(seed=8701, mode=mode)
    print(mode, trained['history'][0], trained['history'][-1])
    print('passes two-part criterion:', trained['memorization_pass'])
```

重点检查small：硬准确率可以很高，为什么交叉熵仍接近log 3？仅以训练准确率作为“优化成功”证据会漏掉什么？再比较He与Xavier：两者都成功，应该如何写一个不夸大的结论？

本实验日志每50步记录一次，但每一步都执行前向、反向与参数更新。日志点不是训练步数。最后一步的指标在完成1200次更新后重新计算，避免报告上一步损失而误称最终值。

9 残差块：分别核对帮助与反例

比较residual_probe(24, 8701, 1.0)与residual_probe(24, 8701, 1/sqrt(24))。模型是 $h + \alpha \tanh(hW)$ ，相加之后没有ReLU，不含BatchNorm，也没有投影跳连。请在报告第一句写出这个具体结构。

手算两个一维反例：分支 $F(h) = -h$ 与 $F(h) = h$ 。计算一块、两块和24块的导数，解释为什么不能把“有恒等路径”说成“总梯度永远不会消失”。然后把前向平方展开，指出哪一项包含输入与分支相关性。

10 练习清单

1. 对四点ReLU例子逐项计算均值、二阶矩、方差、RMS，证明为何只有二阶矩恰好减半。
2. 从 $Z = \sum_i W_i H_i$ 开始，列出所有独立性及零均值条件，推导 $E[Z^2]$ ，解释不能把输入二阶矩直接换成方差。
3. 计算64→32层的Xavier正态标准差、均匀边界和前后向乘子；解释为什么不是两种目标方差的算术平均。
4. 推导ReLU与负斜率0.1的Leaky ReLU对应He方差；预测等宽24层Xavier/ReLU的末层二阶矩。
5. 已知梯度矩阵形状 128×64 ，RMS为0.02，求Frobenius范数；若损失从平均改成求和，梯度和RMS如何变化？
6. 用两个饱和对照解释“输出不小”和“梯度不小”为何不是同一命题，引用至少两个实测数字。
7. 用残差标量反例推翻“恒等路径确保总梯度下界为1”，写出一般块的前向二阶矩展开与结构条件。
8. small配置准确率93.3%、交叉熵约1.0986，是否通过预声明记忆门槛？用一组三分类概率解释。

9. 说明Wine数据来源、抽样规则、预处理责任和泛化边界；设计一个后续泛化实验，不能复用当前训练成绩当测试成绩。
10. 一位同学把学习率乘100000来补偿首层梯度小。列出三个风险，并给出更合理的排查顺序。

验收清单

- 14项unittest正常与-O均通过，有限差分测试真正执行
- 传播实验保留三个权重种子、三种深度、全部八个组合
- 至少解释一处二阶矩与方差的区别、一处范数与RMS的区别
- 四幅图都有单位、横纵轴与零点处理说明
- 30条真实样本来源与许可可追溯，数据哈希已记录
- He三个种子均通过双门槛；Xavier的成功与small的失败均如实保留
- 不把初始化探针冒称训练、不把记忆成功冒称泛化、不把简化残差实验冒称完整ResNet